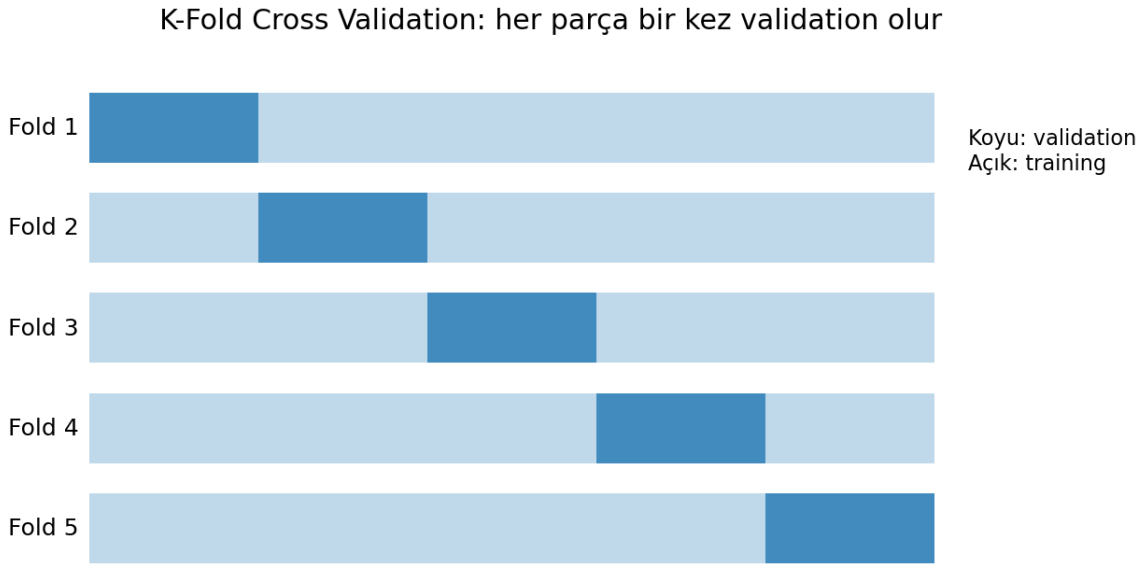
21. CROSS VALIDATION

KFold • StratifiedKFold • GroupKFold • TimeSeriesSplit • Nested CV



Bu modülün amacı
Cross-validation yöntemlerini ezberlemek yerine, gerçek deployment yapısını taklit eden validation tasarımını kurmayı öğrenmek. Doğru CV, model seçiminin güvenilirliğini belirler.

Kazanım	Modül sonunda
KFold	IID problemlerinde fold mantığını ve skor varyansını yorumlayabilirsin.
Stratification	Dengesiz classification verisinde class oranını koruyabilirsin.
Groups	Aynı müşteri/hasta/cihazın train-validation sızıntısını engelleyebilirsin.
Time series	Geleceğin geçmişe sızmadığı temporal evaluation tasarlayabilirsin.
Nested CV	Tuning ile unbiased performance estimation görevlerini ayırabilirsin.
Pipelines	Preprocessing ve feature engineering adımlarını fold içinde fit edebilirsin.

1. Cross Validation neden var?

Tek bir train/validation split model performansının tesadüfi olarak iyi veya kötü görünmesine neden olabilir. Cross-validation, modeli birden fazla validation parçası üzerinde ölçerek performansın ortalamasını ve varyansını görmemizi sağlar.

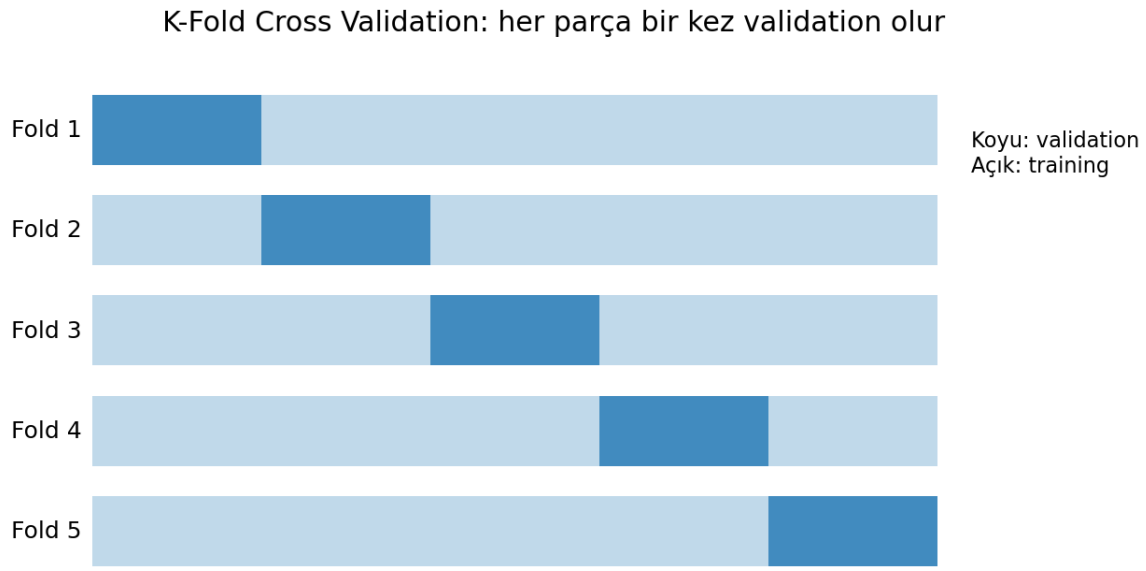
Amaç	Ne kazandırır?
Model selection	Farklı modelleri daha güvenilir karşılaştırma
Hyperparameter tuning	Tek split şansına bağımlılığı azaltma
Uncertainty	Fold-to-fold performans değişimini görme
Data efficiency	Verinin çoğunu training için tekrar tekrar kullanma

CV test set değildir
Cross-validation çoğunlukla model geliştirme ve seçim içindir. Ayrı final test set varsa onu son değerlendirmeye kadar saklamak gerekir.

2. Holdout ile CV farkı

Yöntem	Avantaj	Dezavantaj
Tek holdout	Hızlı ve basit	Split şansına duyarlı
K-Fold CV	Daha stabil performans tahmini	K kez training maliyeti
Repeated CV	Varyansı daha iyi görür	Daha pahalı
Nested CV	Tuning biasını azaltır	Çok pahalı

3. K-Fold Cross Validation



1. Veriyi K parçaya böl.
2. Her iterasyonda 1 fold validation, kalan K-1 fold training olur.
3. K tur sonunda her örnek bir kez validation rolü görür.

4. Metriclerin mean ve standard deviation değerlerini raporla.

```
from sklearn.model_selection import KFold, cross_validate

cv = KFold(n_splits=5, shuffle=True, random_state=42)

scores = cross_validate(
   model, X, y, cv=cv,
   scoring=["neg_mean_absolute_error", "r2"]
)
```

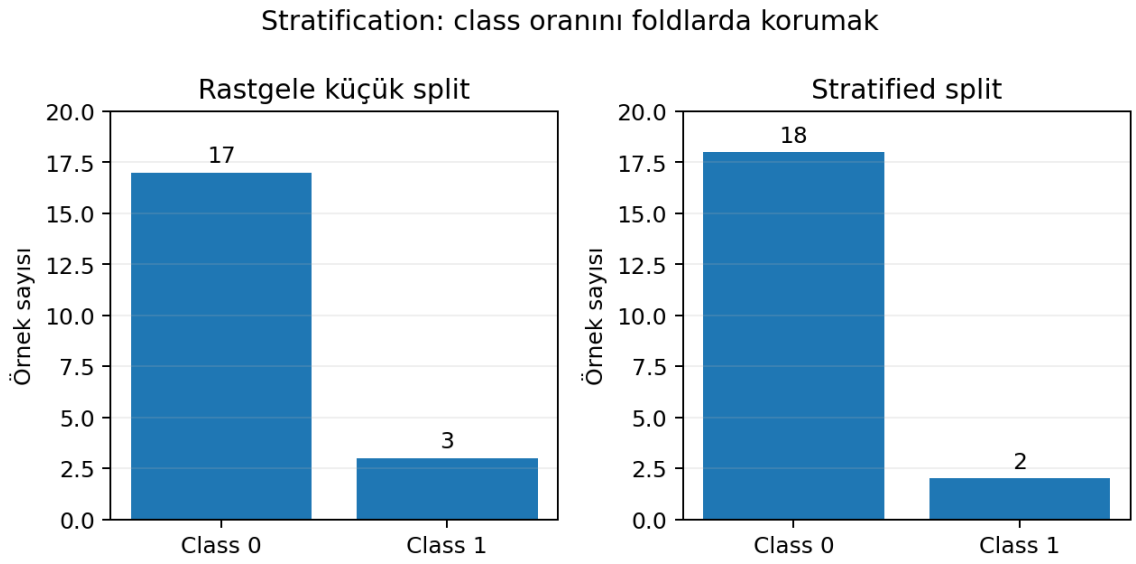
K kaç olmalı?
5 veya 10 yaygın seçimdir. Daha büyük K daha çok training maliyeti ve birbirine daha benzer training setleri üretir; küçük datasetlerde faydalı olabilir.

4. Shuffle ve random_state

IID veride shuffle=True fold kompozisyonunu daha dengeli yapabilir. random_state reproducibility sağlar. Zaman serisinde shuffle kullanmak genellikle yanlıştır.

Reproducibility
Aynı pipeline ve CV splitter ile sonuçların tekrar üretilebilmesi deney yönetimi için kritiktir.

5. StratifiedKFold



Classification problemlerinde özellikle minority class nadirse her foldun class oranını yaklaşık korumak gerekir.

```
from sklearn.model_selection import StratifiedKFold

cv = StratifiedKFold(
   n_splits=5,
   shuffle=True,
```

```
    random_state=42
)
```

Problem	Stratification etkisi
%1 fraud	Her folda fraud örnekleri dağıtır
%8 churn	Fold prevalence farkını azaltır
Balanced class	Yine kullanılabilir ama etkisi daha küçüktür

Stratification her şeyi çözmez
Class oranını korur ama aynı entity veya zaman leakage problemini çözmez.

6. RepeatedStratifiedKFold

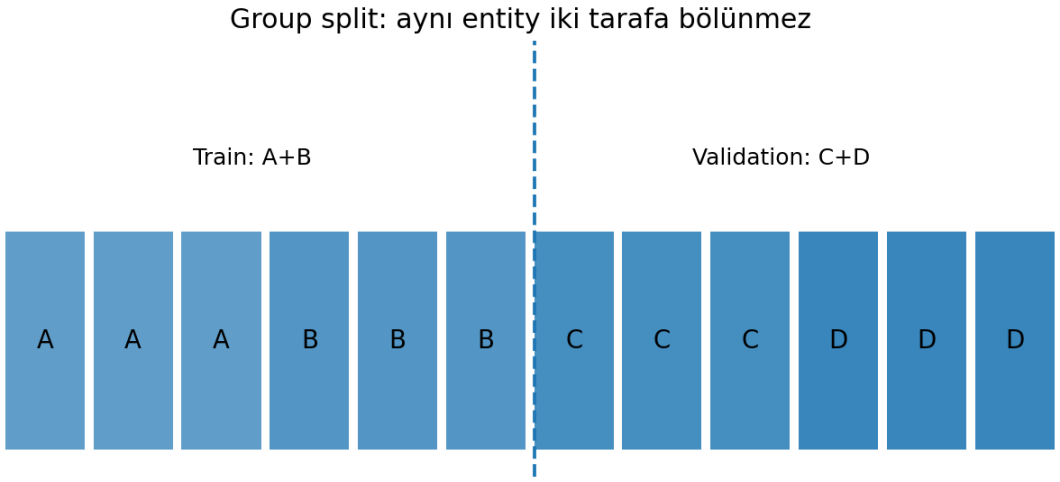
Foldların rastgele oluşumundan gelen belirsizliği görmek için stratified CV birden fazla kez farklı random splitlerle tekrarlanabilir.

```
from sklearn.model_selection import RepeatedStratifiedKFold

cv = RepeatedStratifiedKFold(
    n_splits=5,
    n_repeats=3,
    random_state=42
)
```

Ne raporlanır?
Sadece mean score değil dağılım, standard deviation ve mümkünse confidence interval.

7. GroupKFold



Aynı entity birden fazla satır üretiyorsa random CV aynı müşterinin/hastanın/cihazın kayıtlarını train ve validation tarafına bölebilir. Model entity fingerprintini öğrenerek sahte başarı gösterebilir.

```
from sklearn.model_selection import GroupKFold

cv = GroupKFold(n_splits=5)

scores = cross_validate(
    model, X, y,
    groups=customer_id,
    cv=cv,
    scoring="roc_auc"
)
```

Entity	Group örneği
Hasta	patient_id
Müşteri	customer_id
Cihaz	device_id
Mağaza	store_id
Kullanıcı oturumu	user_id / account_id

Deployment sorusu
Productionda yeni entity tahmin edeceksen group-based evaluation daha gerçekçidir. Aynı entity gelecekte tekrar tahmin edilecekse zaman ve group birlikte düşünülmelidir.

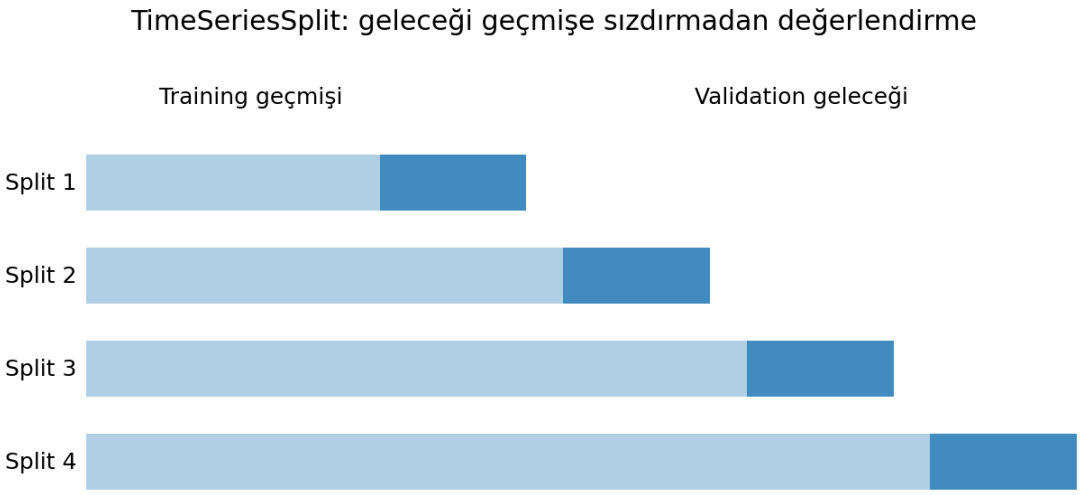
8. StratifiedGroupKFold

Hem class oranını hem group izolasyonunu korumaya çalışır. Örneğin her hastanın birden fazla kaydı ve rare positive outcome varsa faydalıdır.

```
from sklearn.model_selection import StratifiedGroupKFold

cv = StratifiedGroupKFold(
    n_splits=5, shuffle=True, random_state=42
)
```

9. TimeSeriesSplit



Zaman bağımlı problemlerde model geçmiş verilerle eğitilip daha sonraki dönemde doğrulanmalıdır. Random KFold geleceği geçmişe sızdırabilir.

```
from sklearn.model_selection import TimeSeriesSplit

cv = TimeSeriesSplit(n_splits=5)

for train_idx, valid_idx in cv.split(X):
    X_train, X_valid = X.iloc[train_idx], X.iloc[valid_idx]
```

Gerçekçilik
CV şeması production retraining ve prediction akışını taklit etmelidir: hangi tarihte hangi veriyi biliyorduk?

10. Expanding vs rolling window

Window	Mantık	Ne zaman?
Expanding	Training geçmişi her splitte büyür	Tüm geçmiş yararlıysa
Rolling	Sabit yakın geçmiş penceresi	Eski veri relevance kaybediyorsa
Gap / embargo	Train ile validation arasına boşluk	Label gecikmesi veya leakage riski varsa

Concept drift
Eski yılların davranışı artık geçersizse rolling window, expanding windowdan daha gerçekçi olabilir.

11. Time leakage örnekleri

Leakage	Neden yanlış?
Random split ile 2026 kaydı train, 2025 kaydı valid	Gelecek pattern geçmiş tahmine sızıyor
Rolling feature current targetı içeriyor	Target feature içine girmiş
Future aggregate	Prediction sonrasındaki işlemler kullanılıyor
Global scaler tüm dönemlere fit	Future distribution bilgisi train preprocessingine sızıyor

12. Purged / embargo validation fikri

Finans veya overlapping label window problemlerinde train ve validation gözlemlerinin bilgi setleri birbirine taşabilir. Purging overlap eden örnekleri çıkarır, embargo ise zaman tamponu bırakır.

Genel ders
Split yalnız satır indeksine değil, örneklerin bilgi zaman aralığına göre tasarlanmalıdır.

13. Cross-validation ve preprocessing

Scaler, imputer, encoder, feature selector ve target encoder gibi data-driven dönüşümler her foldun yalnız training kısmında fit edilmelidir.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer

pipe = Pipeline([
   ("impute", SimpleImputer(strategy="median")),
   ("scale", StandardScaler()),
   ("model", model)
])

scores = cross_validate(pipe, X, y, cv=cv)
```

Pipeline = leakage bariyeri
Preprocessingi CV dışına çıkarmak validation bilgisini training dönüşümlerine katabilir.

14. Feature selection CV içinde olmalı

Feature selection tüm dataset üzerinde yapılıp sonra CV uygulanırsa validation target bilgisi feature seçim kararına sızar.

```
from sklearn.feature_selection import SelectKBest

pipe = Pipeline([
   ("select", SelectKBest(k=20)),
   ("model", model)
])
```

15. SMOTE / resampling CV içinde olmalı

Oversampling splitten önce yapılırsa sentetik örneklerin türetildiği komşular validation tarafındaki bilgiyle ilişkilenebilir.

Doğru araç
imbalanced-learn Pipeline kullan ve resamplingi yalnız training fold içine yerleştir.

16. Target encoding ve CV

Target encoding en riskli dönüşümlerden biridir. Kategori target meanleri out-of-fold veya fold-aware hesaplanmalıdır.

Yanlış skor patlaması
Naive target encoding özellikle high-cardinality kategorilerde validation skorunu dramatik biçimde şişirebilir.

17. Cross-validation metric seçimi

Problem	Primary CV metric
Regression + lineer iş maliyeti	MAE
Regression + büyük hata pahalı	RMSE
Balanced classification	ROC-AUC/F1 bağlama göre
Rare positive	PR-AUC / recall / precision
Probability quality	Log-loss / Brier
Capacity-based	Precision@k / Recall@k custom scorer

Primary metric önceden
Çok sayıda metric içinde en iyiyi sonradan seçmek selection bias yaratabilir.

18. cross_validate ile birden fazla metric

```
scores = cross_validate(
    model, X, y, cv=cv,
    scoring={
        "roc_auc": "roc_auc",
        "pr_auc": "average_precision",
        "f1": "f1"
    },
    return_train_score=True
)
```

Train vs valid
Train score çok yüksek, validation belirgin düşükse overfitting işareti olabilir.

19. Fold skorlarını nasıl raporlarız?

```
import numpy as np

vals = scores["test_pr_auc"]
print("mean:", vals.mean())
```

```
print("std :", vals.std())
print("folds:", vals)
```

Sadece ortalama yetmez
0.81 ± 0.02 ile 0.81 ± 0.12 aynı güvenilirlikte değildir.

20. Hyperparameter tuning ve CV

GridSearchCV veya RandomizedSearchCV iç döngüde farklı hyperparameterları CV ile karşılaştırır.

```
from sklearn.model_selection import GridSearchCV

search = GridSearchCV(
   pipe,
   param_grid={"model__max_depth": [3,5,8]},
   cv=cv,
   scoring="roc_auc",
   n_jobs=-1
)
search.fit(X_train, y_train)
```

Test set
search.best_score_ model seçimi için CV skorudur; final test score ayrı raporlanır.

21. RandomizedSearch ve Optuna

Parametre uzayı genişse exhaustive grid pahalıdır. RandomizedSearchCV veya Optuna benzeri optimizasyon araçları aynı CV şeması üzerinde daha verimli arama yapabilir.

CV splitter değişmez
Tuning aracını değiştirirken validation mantığını bozma. En önemli bileşen doğru split tasarımıdır.

22. Nested Cross Validation

Nested CV iki döngü kullanır: iç CV hyperparameter seçer, dış CV seçilmiş sürecin unbiased performansını tahmin eder.

Döngü	Görev
Inner CV	Hyperparameter/model selection
Outer CV	Final performance estimation

```
# Kavramsal akış
for outer_train, outer_valid in outer_cv.split(X, y):
   search.fit(X_outer_train, y_outer_train)
   best_model = search.best_estimator_
   score = metric(
      y_outer_valid,
```

```
    best_model.predict(X_outer_valid)
)
```

Ne zaman değerli?
Dataset küçük/orta ve model kıyaslaması ciddi ise. Büyük datasetlerde ayrı holdout test çoğu zaman daha pratik olabilir.

23. Repeated CV vs Nested CV

Yöntem	Çözdüğü problem
Repeated CV	Split rastgeleliğine bağlı variance
Nested CV	Hyperparameter tuningden gelen optimistik bias

24. Learning curves ile CV

Training set boyutu arttıkça train ve validation performansını izlemek, daha fazla veri mi yoksa daha iyi model/feature mı gerektiğini anlamaya yardım eder.

Gözlem	Olası yorum
Train yüksek, valid düşük	High variance / overfit
İkisi de düşük	High bias / underfit
Valid veri arttıkça yükseliyor	Daha fazla veri faydalı olabilir

25. Model karşılaştırma güvenilirliği

İki modelin CV ortalamaları çok yakınsa fold bazlı paired karşılaştırma daha anlamlıdır. Aynı foldlarda iki modelin skor farkını incele.

0.001 fark
Pratik olarak anlamsız veya noise olabilir. Runtime, latency, calibration ve yorumlanabilirlik gibi kriterleri de hesaba kat.

26. Confidence interval ve bootstrap

CV fold sayısı küçük olduğundan klasik confidence interval yorumları dikkat ister. Ayrı test set üzerinde bootstrap, performans belirsizliğini tahmin etmek için yaygındır.

Uncertainty kültürü
Model skorunu tek sayı değil tahmin + belirsizlik olarak görmek daha olgun bir değerlendirme yaklaşımıdır.

27. Gerçek vaka: churn

Aylık churn tahmini yapılan abonelik sisteminde her müşteri birçok ay satır üretir.

Risk	Tasarım
Aynı müşteri iki tarafta	Group veya time-aware split
Gelecek ay bilgisi featureda	Point-in-time feature generation
Class imbalance	Stratification veya appropriate metrics
Concept drift	Out-of-time test

En gerçekçi tasarım
Eski aylarla train, sonraki aylarla validate; aynı prediction horizon ve feature cutoff mantığını production ile eşleştir.

28. Gerçek vaka: fraud

Fraud davranışı zamanla değişir ve aynı kart/kullanıcı çok sayıda işlem üretir.

- Random CV sahte kolaylık yaratabilir.
- Time split + entity leakage kontrolleri gerekli.
- PR-AUC ve precision@review_capacity izlenmeli.
- Label gecikmesi varsa embargo/gap düşünülmeli.

29. Gerçek vaka: medical imaging

Bir hastanın birden fazla görüntüsü varsa görüntü düzeyinde random split aynı hastayı train ve validationa taşır.

Doğru group
patient_id üzerinden group split. Aksi halde model hasta/spesifik acquisition özelliklerini öğrenebilir.

30. Gerçek vaka: store sales

Birden fazla mağazada satış tahmini yapılırken deployment hedefi yeni mağaza mı, mevcut mağazanın geleceği mi olduğuna göre split değişir.

Deployment hedefi	Evaluation
Mevcut mağazanın geleceği	Time split
Yeni mağaza generalization	Group/store holdout
İkisi birlikte	Group + time senaryolarını ayrı test et

31. CV anti-patternleri

- Preprocessingi tüm dataset üzerinde fit edip sonra CV yapmak.
- Time data üzerinde shuffle KFold kullanmak.

- Aynı kullanıcıyı train ve validationa bölmek.
- Target encodingi validation target ile hesaplamak.
- SMOTEu CVden önce uygulamak.
- CV skoruna bakıp metric veya split şemasını tekrar tekrar değiştirmek.
- Fold standard deviationını saklamak.
- Test seti hyperparameter seçiminde kullanmak.
- Production prediction horizonını CVde taklit etmemek.
- Küçük skor farklarını kesin üstünlük gibi sunmak.

32. CV seçim karar ağacı

Soru	Evetse
Zaman sırası önemli mi?	TimeSeriesSplit / temporal holdout
Aynı entity birden çok satır mı?	GroupKFold / group holdout
Classification dengesiz mi?	Stratified splitter
Hem group hem imbalance mı?	StratifiedGroupKFold
IID ve bağımsız satırlar mı?	KFold / StratifiedKFold
Tuning biasını da ölçmek mi istiyorsun?	Nested CV

33. Sklearn kod şablonları

```
# Regression IID
KFold(n_splits=5, shuffle=True, random_state=42)

# Classification IID
StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Group data
GroupKFold(n_splits=5)

# Classification + group
StratifiedGroupKFold(n_splits=5, shuffle=True, random_state=42)

# Time data
TimeSeriesSplit(n_splits=5)
```

34. Custom CV splitter düşüncesi

Gerçek iş problemleri bazen hazır splitters ile tam ifade edilemez. Örneğin 30 günlük label window, 7 günlük embargo, müşteri group izolasyonu ve aylık retraining birlikte gerekebilir.

İleri seviye prensip
Custom splitter yazarken her validation örneğinin productionda karşılaşacağı bilgi setini birebir simüle etmeye çalış.

35. CV sonuçlarını deney tablosuna dönüştürmek

Model	Mean PR-AUC	Std	Fit time	Not
Logistic	0.421	0.012	1x	Strong baseline
Random Forest	0.447	0.019	6x	Higher variance
LightGBM	0.481	0.010	3x	Best mean/stability

Seçim
Sadece en yüksek mean değil variance, training time, latency, calibration ve business fit birlikte değerlendirilmeli.

36. Reproducible CV deneyleri

- random_state sabitle.
- Dataset/version kaydet.
- Feature pipeline version kaydet.
- CV splitter config kaydet.
- Primary metric ve scorer kodunu versionla.
- Library versionlarını kaydet.

37. Mülakat soruları

Soru	Kısa cevap
K-Fold CV ne yapar?	Her fold bir kez validation olur; skorlar ortalanır.
StratifiedKFold neden?	Classification class oranını foldlarda yaklaşık korur.
GroupKFold ne zaman?	Aynı entitynin train/validationa bölünmemesi gerektiğinde.
TimeSeriesSplit neden shuffle yapmaz?	Gelecekten geçmişe leakage oluşmaması için.
CV içinde scaler neden?	Validation distribution bilgisini preprocessing fitine sızdırmamak için.
Nested CV nedir?	İç döngü tuning, dış döngü unbiased evaluation.
Repeated CV ne sağlar?	Split randomlığı kaynaklı variancei daha iyi görür.
SMOTE nerede yapılmalı?	Yalnız training fold içinde.
Target encoding nasıl yapılmalı?	Out-of-fold/fold-aware.
K=LOOCV her zaman iyi mi?	Hayır; pahalı ve variance özellikleri farklı olabilir.
CV std neden önemli?	Model performans stabilitesini gösterir.
Random CV time data için neden kötü?	Future pattern train tarafına sızabilir.
Group ve time ikisi de varsa?	Deploymentı taklit eden combined/custom split gerekebilir.
Test set yine gerekli mi?	Mümkünse final unbiased evaluation için evet.
CV skorları çok yakınsa?	Uncertainty, paired fold farkları ve operational trade-offları incele.

38. Uygulama alıştırmaları

1. Iris veya benzer classification datasetinde KFold ve StratifiedKFold skorlarını karşılaştır.
2. Imbalanced dataset üret ve folds içindeki prevalence dağılımını göster.
3. Aynı müşteriden 5 satır üreten sentetik veriyle random CV vs GroupKFold farkını ölç.
4. Trend içeren zaman datasında random KFold ile TimeSeriesSplit performans farkını göster.
5. Scaleri CV dışına çıkararak leakage örneği oluştur.

6. SMOTEu CV öncesi ve fold içinde uygulayıp skor farkını karşılaştır.
7. GridSearchCV ile nested CV kur ve naive CV score ile farkını ölç.
8. RepeatedStratifiedKFold ile skor dağılımını histogram olarak çiz.
9. Fold bazlı iki model skor farklarını raporla.
10. Custom business scorer ile CV yap.

39. Mini portföy projesi

Proje: Validation Design Lab - Leakage-Safe Model Selection

Çıktı	İçerik
01_iid_cv.ipynb	KFold / StratifiedKFold benchmark
02_group_cv.ipynb	Customer-level leakage demonstration
03_time_cv.ipynb	Temporal validation and drift
04_nested_cv.ipynb	Hyperparameter tuning bias
05_pipeline_leakage.ipynb	Scaler/encoder/SMOTE leakage tests
src/splitters.py	Reusable CV configs/custom splitter
report.md	Validation rationale, score uncertainty and recommendations

CV cümlesi
Designed leakage-safe cross-validation strategies for IID, grouped and temporal datasets, including nested hyperparameter evaluation and reproducible model-selection reporting.

40. Final cheat sheet

Kavram	Hatırlanacak fikir
Holdout	Hızlı ama split şansına duyarlı
KFold	IID genel CV
StratifiedKFold	Class oranını koru
GroupKFold	Entity leakage engelle
StratifiedGroupKFold	Group + class balance
TimeSeriesSplit	Geçmişten geleceğe validation
Repeated CV	Split varianceını gör
Nested CV	Tuning biasını ayır
Pipeline	Preprocessing fold içinde fit
OOF encoding	Target leakage önle
Mean + std	Skor + stability
Final test	Son unbiased kontrol

Bu modülden sonra
Hyperparameter Optimization bölümünde arama yöntemlerini doğru validation tasarımı üzerine kurarak gerçek model seçimi yapabileceksin.